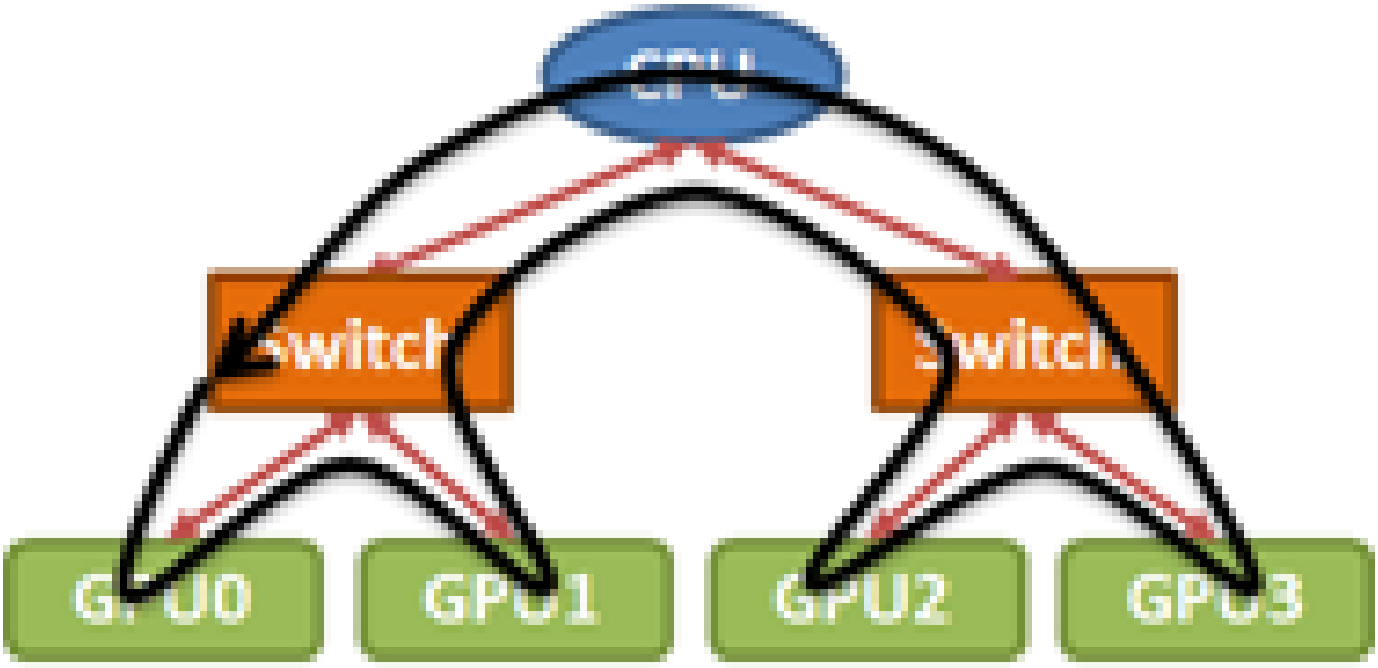
Fast Multi-GPU collectives with NCCL



Apr 07, 2016

+2 Like Discuss (14)

By Nathan Luehr

Today many servers contain 8 or more GPUs. In principle then, scaling an application from one to many GPUs should provide a tremendous performance boost. But in practice, this benefit can be difficult to obtain. There are two common culprits behind poor multi-GPU scaling. The first is that enough parallelism has not been exposed to efficiently saturate the processors. The second reason for poor scaling is that processors exchange too much data and spend more time

NCCL (pronounced “Nickel”) is a library of multi-GPU collective communication primitives that are topology-aware and can be easily integrated into your application. Initially developed as an open-source research project, NCCL is designed to be light-weight, depending only on the usual C++ and CUDA libraries. NCCL can be deployed in single-process or multi-process applications, handling required inter-process communication transparently. Finally, the API will be very familiar to anyone with experience using MPI’s collectives.

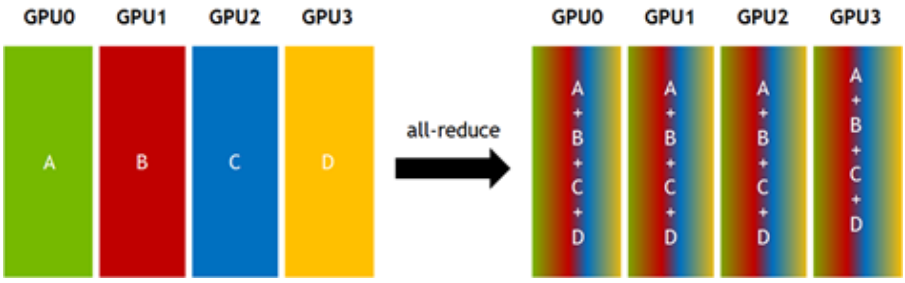


Figure 1: Illustration of the All-Reduce collective.

Collective Communication

Collective communication routines are common patterns of data transfer among many processors. If you have experience with MPI, then you are probably already familiar with several collective operations. For example, all-reduce starts with independent arrays V_k of *N* values on each of *K* processors and ends with identical arrays *S* of *N* values, where $S[k] = V_0[k] + V_1[k] + ... + V_k[k]$, for each processor *k*. See Figure 1.

Another common collective is all-gather, where each of K processors begins with an independent array of *N* values, and collects data from all other processors to form a result of dimension *N*K*, as Figure 2 shows.

Figure 2: Illustration of the All-Gather collective.

Figure 3: Illustration of the Broadcast collective.

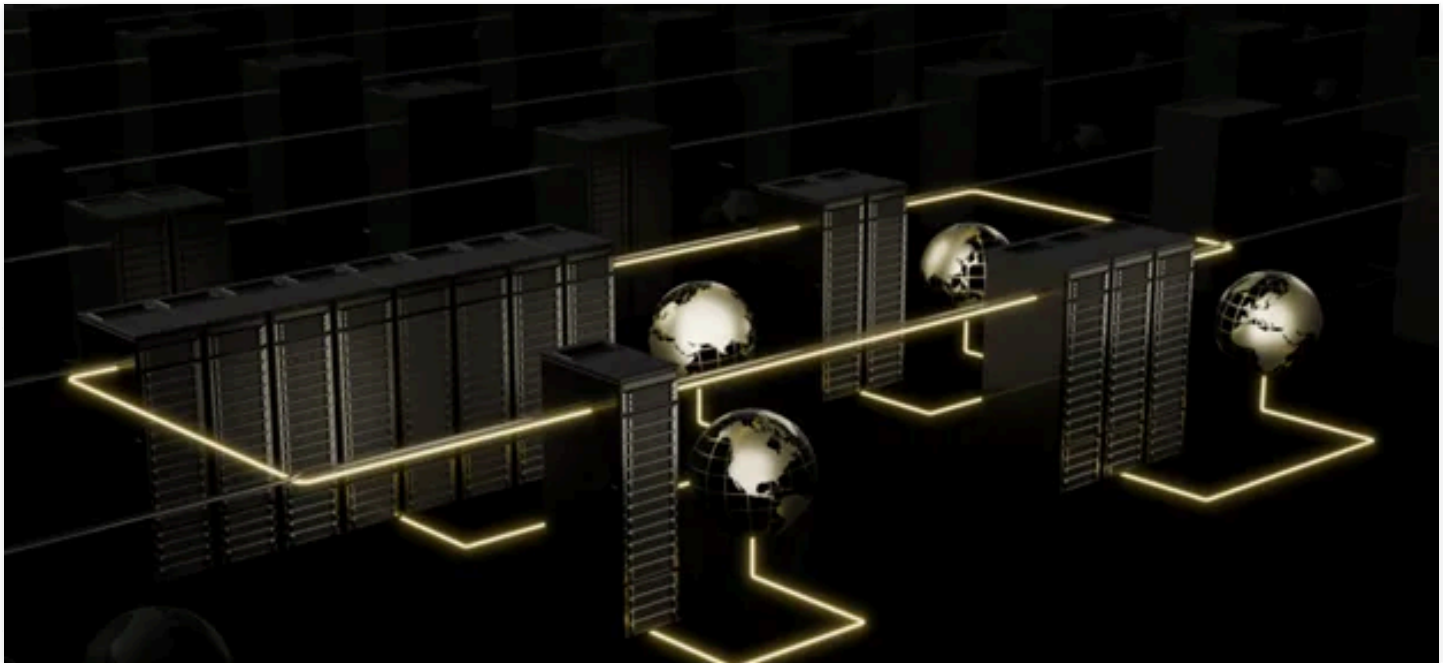
All of the above collectives can be performed either “out-of-place,” with separate input and output buffers, or “in-place,” with overlapping input and output.

There are numerous approaches to implementing collectives efficiently. However, it is critical that our implementation takes the topology of interconnects between processors into account. For example, consider a broadcast of data from GPU0 to all other GPUs in the PCIe tree topology pictured below.

Figure 4: A common PCIe topology for 4 GPUs attached to a single CPU. Red arrows represent PCIe x16 connections.

A two-step tree algorithm is a common choice in this situation. In the first step the data is sent from the GPU0 to a second GPU, and in the second step both of these send data to the remaining processors. However, we have a choice. We can either send data from GPU0 to GPU1 in the first step and then GPU0 to GPU2 and GPU1 to GPU3 in the second, or we can perform the initial copy form GPU0 to GPU2 and then GPU0 to GPU1 and GPU2 to GPU3 in the second step. Examining the topology, it is clear that the second

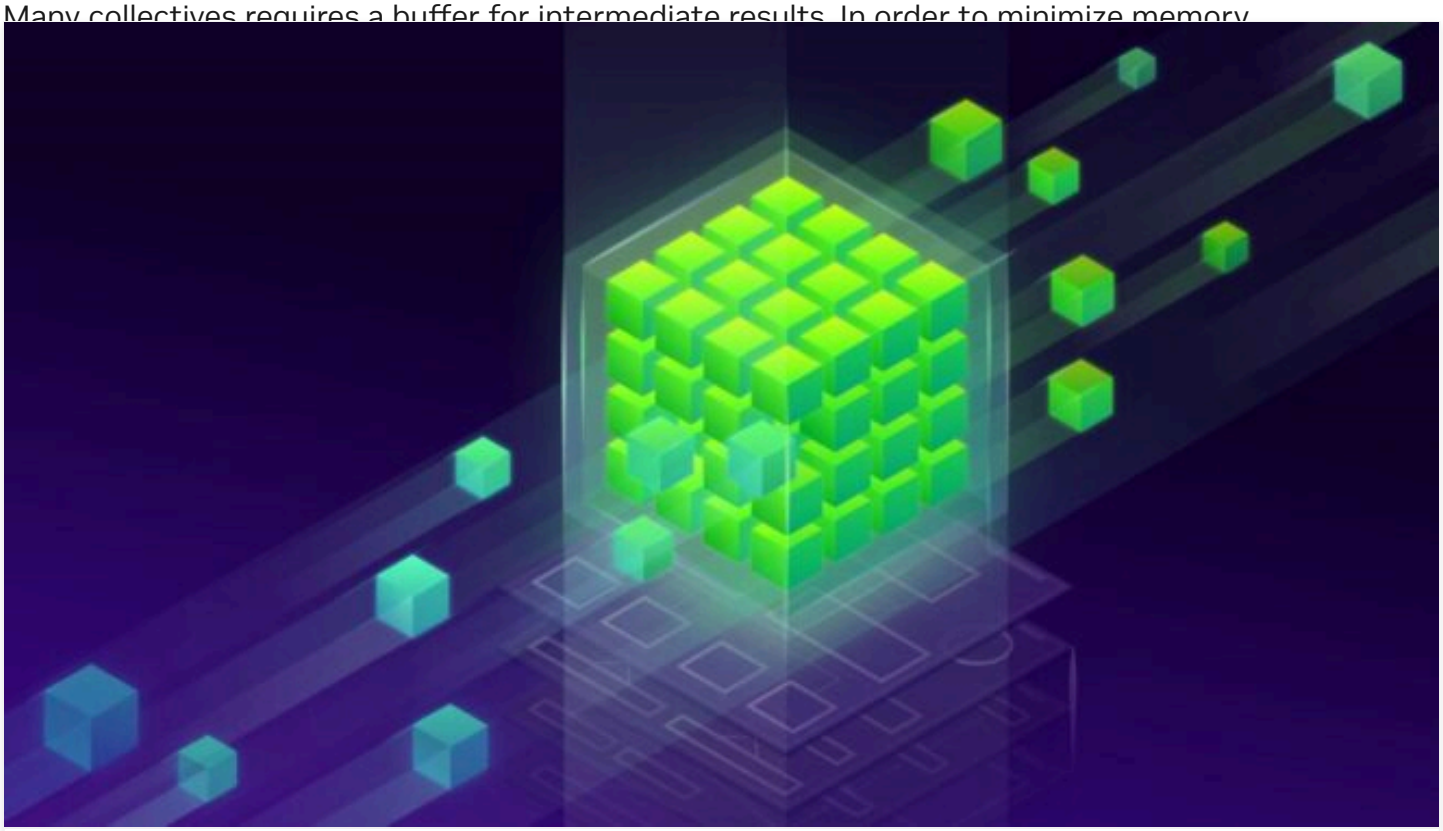
Related posts



Fusing Communication and Compute with New Device API and Copy Engine Collectives in NVIDIA NCCL 2.28

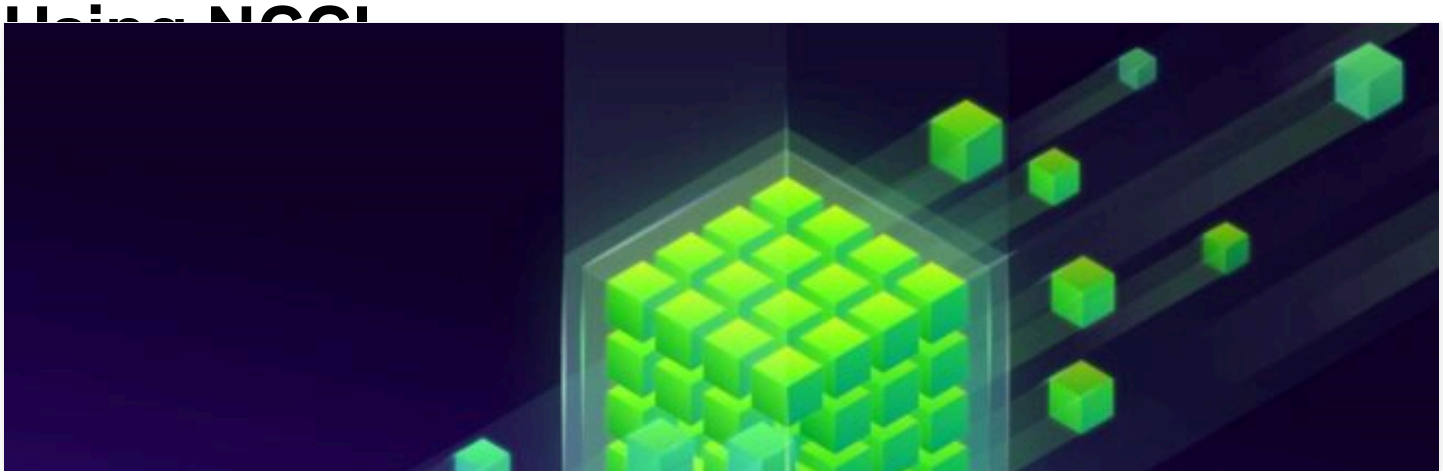
performance for your application while freeing you from having to worry about specific hardware configurations.

Many collectives requires a buffer for intermediate results. In order to minimize memory



New Scaling Algorithm and Initialization with NVIDIA Collective Communications Library 2.23

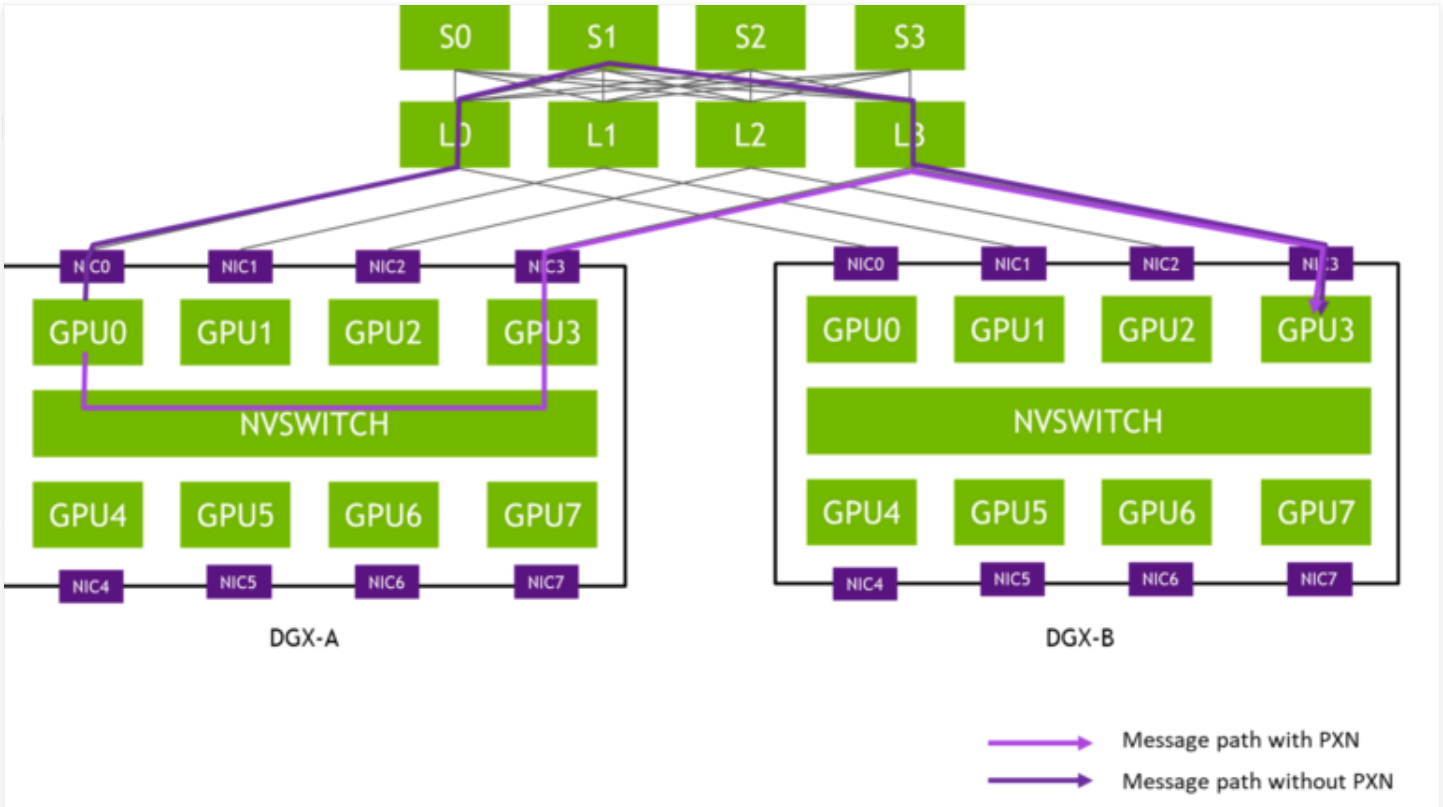
Using NCCL





Memory Efficiency, Faster Initialization, and Cost Estimation with NVIDIA Collective Communications Library 2.22

ranks of the clique to find each other. The same `cliqueId` must be used by all ranks. To achieve



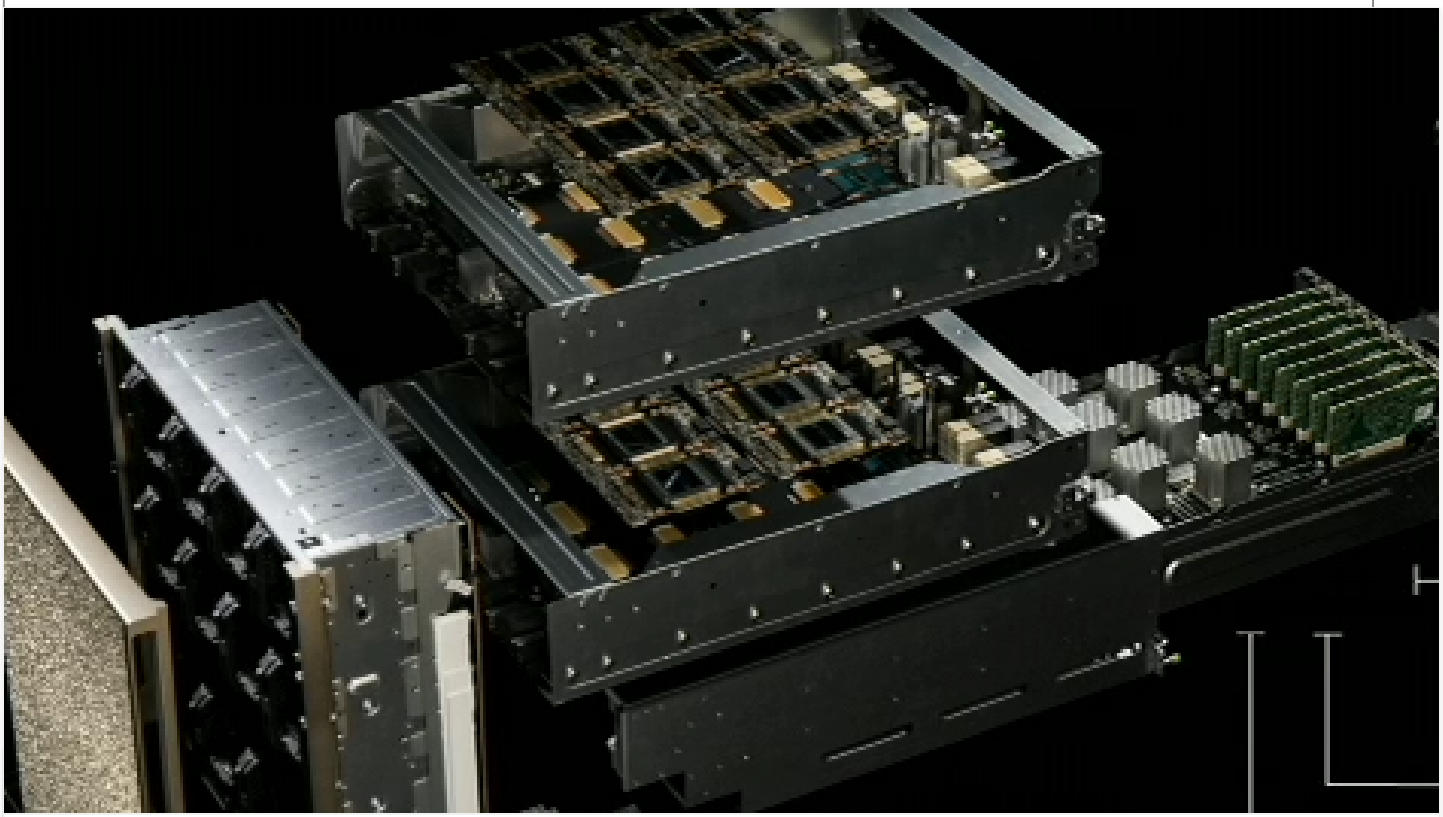
Doubling all2all Performance with NVIDIA Collective Communication Library 2.12

initialize NCCL in an otherwise single-threaded application.

```
ncclResult_t ncclCommInitAll(ncclComm_t* comms,
                             int nGPUs,
                             int* devList);
```

the communicator object initialized, you can call the collectives through their host functions, such as `ncclAllReduce()`.

```
ncclResult_t ncclAllReduce(void* sendoff,
                           void* recvbuff,
                           int count,
                           ncclDataType_t type,
                           ncclRedOp_t op,
                           ncclComm_t comm,
```



Scaling Deep Learning Training with NCCL

Performance

The performance that NCCL collectives achieve depends on the exact topology of the computer. In the best case, all GPUs share peer access. This is most common for workstations equipped with a few GPUs. Larger servers usually sport dual CPUs, with some GPUs segregated on different IO hubs. Figure 6 shows NCCL bandwidth for various collectives measured on an NVIDIA Digits DevBox equipped with 4 GeForce GTX Titan X GPUs arranged as in Figure 4 above.

Figure 6: Link bandwidth achieved by various NCCL collectives.

The red bar at 10.4 GB/s represents the bandwidth achieved by a large cudaMemcpy between two of the GPUs (specifically GPU 0 and 2 in Figure 4). NCCL sustains a high percentage of this peak bandwidth while performing communication among all four GPUs.

Future Directions & Final Remarks

The goal of NCCL is to deliver topology-aware collectives that can improve the scalability of your multi-GPU applications. By using NCCL you can get great performance without having to think about low-level hardware details.

Simple code examples for both single-process and MPI applications are distributed with NCCL. As a research project, we welcome your feedback as we continue to evolve the project. Please try it out and let us know what you think! For an update on the latest developments, come see my NCCL talk at GTC.

Discuss (14) +2 Like

Tags

Simulation / Modeling / Design | CUDA | NCCL | MPI | Multi-GPU | Supercomputing

About the Authors

Comments

Notable Replies



April 18, 2016

anon23970270

Hi Nathan, I've looking forward to this project to come out!
I have a question: If I want to perform a 2D domain decomposition (using a Cartesian domain for simplicity) with NCCL, how would you advice to perform the halo regions exchange using only the 5 collectives in the project?



April 20, 2016

anon92716381

This is an important library for our work. We use numerous multi-GPU applications. Having a topology-aware library should ease our programming burden.



November 16, 2016

anon94217786

Does this tool support the P8NVLink system with P100 GPU? From the blog below, this tool need the GPUDirect but OpenPOWER system doesn't have GPUDirect. That is why I ask if this tool is ready for OpenPOWER P8NVLink system with P100 GPU.
Nvidia blog
==========
https://devblogs.nvidia.com...
"NCCL makes extensive use of GPUDirect Peer-to-Peer direct access to push data between processors."



November 18, 2016

anon5634387

NCCL will automatically fall back to pinned buffers if P2P direct access is not available. That said, the GitHub version of NCCL is not optimized for NVLink, so the achieved performance will

DEVELOPER

Join



anon19982686

Does NCCL provide benefits when using a heterogeneous set of NVIDIA cards or must they be a homogeneous matched set? For example, will I see benefits if I have a GTX 980ti and GTX 1070 in my machine?

Continue the discussion at forums.developer.nvidia.com

9 more replies

Participants



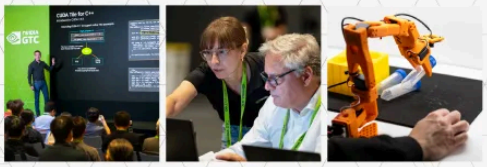




Learn. Build. Get Certified.

Experience hands-on workshops, training labs, and certification at GTC Berlin.

Learn More



NVIDIA GTC

Sign up for NVIDIA News

Subscribe Follow NVIDIA Developer

Privacy Policy | Your Privacy Choices | Terms of Use | Accessibility | Corporate Policies | Contact

Copyright © 2026 NVIDIA Corporation